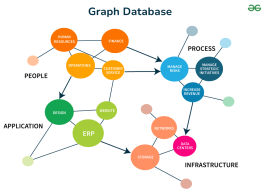
RPG Game Database Design

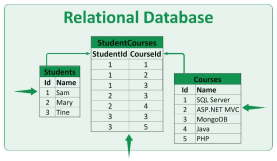
Version 2

Group: Bajls

Ahmad Abdel Razak Hussein Alkaseb
Benjamin Sebastian Barrales Hernandez
Jeppe Ronning Koch
Laith Abdel Razak Hussein Alkaseb

Course	Database
Project	RPG Game by Bajls
Group number	Bajls
Date of delivery	22/5/2026
List of figures	10





List of Figures

Contents

1. Introduction

Our goal is to design and develop a next-generation RPG game inspired by the creative freedom and social interaction found in platforms like Roblox. The game allows players to explore an interactive world, customize their characters in detail, and engage in dynamic gameplay experiences.

At its core, the game revolves around a customizable character that can move freely in a large open world. Players experience the game through their character, whose appearance, identity, and status are visible to others in real time. As in Roblox-inspired games, visual identity and personalization are central to the player experience.

The first development phase focuses on character customization and player identity. Players create a personal profile and design one or more unique characters with specific physical traits and attributes. These traits define how the character looks and can later support social interactions and additional gameplay mechanics.

The world is structured around exploration, housing, garages, vehicles, quests, drugs, and social systems such as gangs. Every character is stored in a structured database that maintains logical relationships between profiles, characters, and owned properties. The system enforces clear rules for ownership, identity, and mandatory attributes.

The player experience includes:

- Creating a personal profile
- Customizing characters with physical traits
- Exploring the map
- Owning a house
- Optionally joining a gang
- Interacting with other players

The game architecture must support both regular players and administrators who manage the system.

1.1. System overview and cloud architecture

This project is organized as a Java REST API with three database implementations: PostgreSQL, MongoDB, and Neo4j. PostgreSQL is the relational source database, and the migration application can move the same domain data into MongoDB and Neo4j.

Architecture diagram (cloud deployment)

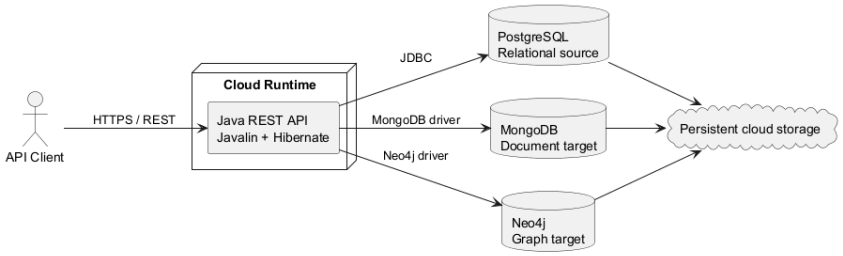


Figure 1: Cloud deployment architecture

```
[Client / Browser / API Consumer]
                |
                v
       [Java Application Service]
       (REST API + migration)
          |        |        |
          v        v        v
   [PostgreSQL] [MongoDB] [Neo4j]
          |        |        |
          v        v        v
     [Persistent Cloud Storage]
```

In cloud environments, the application service and databases run as separate managed workloads. Each database persists data on dedicated storage, while the Java application contains the REST API and migration logic.

Local development deployment (`docker-compose`)

The local setup uses `docker-compose.yml` in the project root to start application and database services in one command.

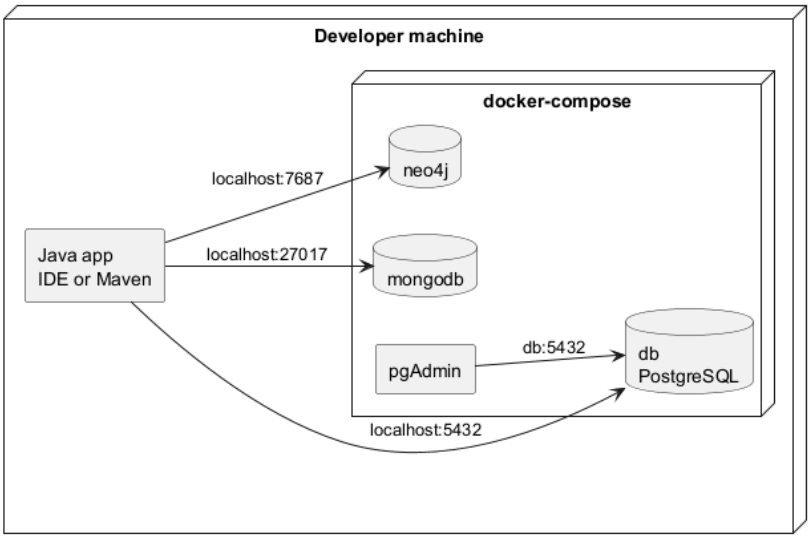


Figure 2: Local development deployment using docker-compose

```
docker-compose.yml
    |
    +-- db (PostgreSQL source database)
    +-- mongodb (document database target)
    +-- neo4j (graph database target)
    +-- pgadmin (PostgreSQL administration UI)
    |
    +-- Java app runs from IDE/Maven and connects to the services
```

Typical local startup command:

```
docker compose up -d
```

After the containers are running, the Java application is started from the IDE or Maven. On startup it resets PostgreSQL, reseeds it from `sqls/seed.sql`, migrates the fresh snapshot into MongoDB and Neo4j, and then starts three HTTP servers.

1.2. Explanation of choices for databases and programming languages, and other tools

The project uses **PostgreSQL**[1], **MongoDB**[2], and **Neo4j**[3] to compare the same RPG domain across relational, document, and graph database paradigms.

PostgreSQL is used because the domain is strongly relational and depends on strict integrity rules. It gives stable transactional behavior, foreign-key enforcement, and support for normalized schemas with junction tables such as `gang_affiliations`.

MongoDB is used because profile-centered reads can be represented as one document containing embedded characters, houses, garages, vehicles, and relationship metadata. This demonstrates denormalized aggregate modeling.

Neo4j is used because gangs, quests, ownership, and character relations are naturally traversable as nodes and relationships. It demonstrates how relationship-heavy domains can be queried without join tables.

The application is implemented in **Java 17**[4] with **Maven**[5] as the build tool. Java was selected because the project team works with an object-oriented domain model, and Java integrates directly with JPA annotations for entity mapping. Maven provides predictable dependency management and reproducible builds across environments.

For persistence, the project uses **Hibernate (JPA)**[6]. This allows the team to model business entities (`Profile`, `GameCharacter`, `House`, `Garage`, `Vehicle`, `Drug`, `Quest`, `Gang`, and the relationship entities) directly in code and keep the SQL schema aligned through mapping metadata. Hibernate is configured for PostgreSQL and supports both local development and deployment profiles through environment variables.

Additional tools include:

- **Lombok** for reducing boilerplate code in entities (getters, setters, constructors, builders).
- **Testcontainers** for disposable PostgreSQL instances during tests, which improves repeatability and reduces local setup variance.
- **pgAdmin** for visual inspection of the physical schema and foreign key network.

[1] PostgreSQL Documentation: https://www.postgresql.org/docs/
[2] MongoDB Documentation: https://www.mongodb.com/docs/
[3] Neo4j Documentation: https://neo4j.com/docs/
[4] Java 17 Documentation (Oracle): https://docs.oracle.com/en/java/javase/17/
[5] Apache Maven Documentation: https://maven.apache.org/guides/
[6] Hibernate ORM Documentation: https://hibernate.org/orm/documentation/

HTTP Request Files for API Testing

Rather than adopting external API testing tools like Postman, the project uses native HTTP request files (`*.http`) directly within the development environment. This lightweight approach provides several advantages:

- **Minimal tooling overhead**: HTTP files are plain-text files that can be version-controlled in Git, eliminating the need to learn and manage a separate desktop application.
- **Built-in IDE support**: Modern code editors (e.g., VS Code with REST Client extensions) natively execute HTTP requests, offering seamless integration with the development workflow.
- **Standardized format**: HTTP request files are portable and maintainable across team members.

The project includes three HTTP request files:

- **`mongodb.http`** – Contains test requests for MongoDB queries, document operations, and validation of the document-centric data model.
- **`neo4j.http`** – Contains test requests for Neo4j graph queries, traversals, and relationship-based operations on gang and quest networks.
- **`postgres.http`** – Contains test requests for PostgreSQL operations, including profile creation, character management, and relational constraint validation.

These files serve as executable documentation and rapid testing aids, allowing developers to validate database behavior and API contracts without context-switching to external applications. They also facilitate team collaboration by embedding request templates and expected response patterns directly in the codebase.

The next chapter turns these technology choices into concrete database rules and schema decisions.

---

2. Relational database

This section defines the functional requirements for our RPG game inspired by Roblox. The purpose is to clearly define persons, profiles, characters, and their relationships in the system. The system must enforce strict rules to ensure data integrity, logical consistency, and reliable gameplay functionality.

To keep a clear thread through the chapter, we move in this order: requirements -> conceptual/logical design -> normalization -> physical implementation -> stored database objects -> realistic seed data -> access control.

2.1. Intro to relational databases

A relational database organizes information in tables connected by keys. This model is suitable for the RPG domain because core business rules are relationship-based: one profile can own many characters, each character must belong to exactly one profile, and gang membership is many-to-many through a junction table.

Relational systems are also appropriate when consistency is more important than schema flexibility. In this project, invalid states such as characters without profiles, houses without owners, or duplicate membership records must be prevented in the database itself. PostgreSQL handles this through primary keys, foreign keys, unique constraints, and transaction guarantees.

2.2. Database design

The database design follows a layered progression from requirements to implementation:

- First, domain requirements define mandatory entities.
- Next, the conceptual and logical models translate those rules into normalized relations.
- Finally, the physical model implements data types, keys, and constraints in PostgreSQL/Hibernate.

This process ensures that design decisions are traceable from business rules to concrete table definitions and mapping annotations.

Profile Requirements

Person and Profile

- A person can have exactly one profile in the system.
- The profile represents the player's digital identity.
- A profile must be uniquely identifiable by `email` and `username`.
- A profile stores exactly one role value: `USER` or `ADMIN`.

This keeps account identity simple and gives the application one clear place to enforce login and authorization rules.

Character Requirements

Profile to Character Relationship

- A profile can own one or more characters.
- A character must belong to exactly one profile.
- A character cannot exist without being linked to a profile.

This creates a one-to-many relationship between `Profile` and `Character`.

Character Attributes Each character must have exactly one value for the required appearance and classification fields:

- one gender
- one skin color
- one eye color
- one height value
- one weight value

In the new model these values are stored directly on `characters` instead of separate lookup tables. The values are still controlled through application enums and SQL `CHECK` constraints.

Housing and Garage Requirement

- A character must have exactly one house.
- A character must have exactly one garage.
- A house belongs to exactly one character.
- A garage belongs to exactly one character.
- A garage can contain zero or more vehicles.

This creates two mandatory one-to-one relationships from `Character` to `House` and `Garage`, and a one-to-many relationship from `Garage` to `Vehicle`.

Quest, Drug, and Gang Requirement

- Gang membership is optional.
- A character can belong to zero or more gangs.
- A character can have zero or more quests.
- A character can hold zero or more drugs.

These are modeled as many-to-many relationships in the domain, with membership-specific data stored in junction tables in the relational implementation.

Data Integrity Rules The system must enforce the following constraints:

- Every person has at most one profile.
- Every profile has exactly one role.
- Every character belongs to exactly one profile.
- Every character has all required appearance values.
- Every character has exactly one house and one garage.
- A garage can contain multiple vehicles.
- Duplicate quest assignments, drug links, and gang memberships must be prevented for the same character pair.

Relationship Summary

- `Profile` to `Character`: One-to-Many
- `Character` to `House`: One-to-One (Mandatory)
- `Character` to `Garage`: One-to-One (Mandatory)
- `Garage` to `Vehicle`: One-to-Many
- `Character` to `Drug`: Optional Many-to-Many
- `Character` to `Quest`: Optional Many-to-Many
- `Character` to `Gang`: Optional Many-to-Many

Conclusion These requirements define the new structural foundation of the RPG system. Identity stays centered on profiles, gameplay stays centered on characters, and assets and activities are represented explicitly through houses, garages, vehicles, quests, drugs, and gangs.

2.2.1. Entity/Relationship Model (Conceptual -> Logical -> Physical model)

This section presents the entity/relationship progression from conceptual understanding to logical structure, and prepares the transition to the physical implementation in Section 2.3.

Conceptual Model The conceptual model describes the business domain without technical implementation details such as data types or SQL syntax. Its purpose is to show what information the system must manage and how core concepts are related from a game and business perspective. In this project, `Characters` is the central concept because most game actions, ownership rules, and social interactions are represented through character entities.

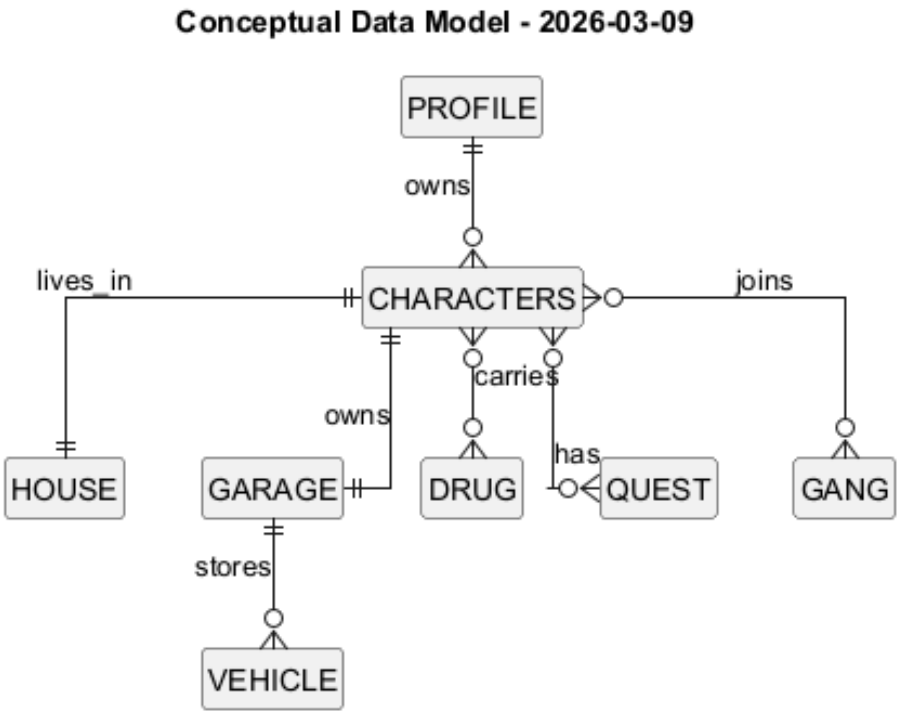


Figure 3: Conceptual data model. Authoritative source: 'conceptual-uml-03092026.puml'

Conceptual Diagram The model includes the following core entities:

- **Profiles**: the persistent player identity used for login and account-level ownership.

- **Characters**: playable identities controlled by profiles inside the game world.
- **Houses**: character-owned properties used to represent private assets and ownership constraints.
- **Garages**: character-owned storage locations for vehicles.
- **Vehicles**: assets stored inside a garage.
- **Drugs**: tradable or collectible items related to a character.
- **Quests**: tasks that can be assigned to one or more characters.
- **Gangs**: social groups that support optional membership and multi-character affiliation.

Key Relationships in the Diagram

- `Profiles` to `Characters`: one-to-many. A profile can own multiple characters, and each character belongs to one profile.
- `Characters` to `Houses`: one-to-one (mandatory). Each character must be linked to one house, and each house belongs to one character.
- `Characters` to `Garages`: one-to-one (mandatory). Each character has exactly one garage, and each garage belongs to one character.
- `Garages` to `Vehicles`: one-to-many. A garage can store zero or more vehicles.
- `Characters` to `Drugs`: optional many-to-many.
- `Characters` to `Quests`: optional many-to-many.
- `Characters` to `Gangs`: optional many-to-many. A character may join zero or more gangs, and each gang may contain multiple characters.

Cardinality and Modality In this project, relationship rules are described using both **cardinality** and **modality**:

- **Cardinality** defines the maximum number of related rows (for example one-to-one, one-to-many, many-to-many).
- **Modality** defines whether participation is mandatory or optional (minimum 1 or minimum 0).

Applied to our model:

- `Profile` -> `Character`: cardinality is one-to-many, modality is mandatory on `Character` side (every character must have one profile) and optional on profile side (a profile can exist with zero characters).
- `Character` -> `House`: cardinality is one-to-one, modality is mandatory on character side.
- `Character` -> `Garage`: cardinality is one-to-one, modality is mandatory on character side.
- `Garage` -> `Vehicle`: cardinality is one-to-many, modality is optional on the vehicle side because a garage may be empty.
- `Character` <-> `Drug`, `Quest`, `Gang`: cardinality is many-to-many, modality is optional on both sides.

The conceptual model also defines participation rules. Participation is mandatory for `Character` to `Profile`, `Character` to `House`, and `Character` to `Garage`, because these are required for valid gameplay state. Participation is optional for gangs, quests, drugs, and vehicles.

From a domain perspective, this model prevents ambiguous ownership. A profile owns characters, characters own houses, and social membership is kept independent through gangs. This separation is important because each area has different lifecycle rules: deleting or deactivating a profile affects owned characters, while gang membership can be added or removed without changing character identity.

The conceptual view therefore acts as the foundation for the next models. It communicates business meaning to both technical and non-technical stakeholders, verifies that rules are complete before implementation, and reduces redesign risk later in the project.

Logical Model The logical model translates the conceptual design into a normalized relational structure. At this level business rules are transformed into enforceable structural constraints with attributes.

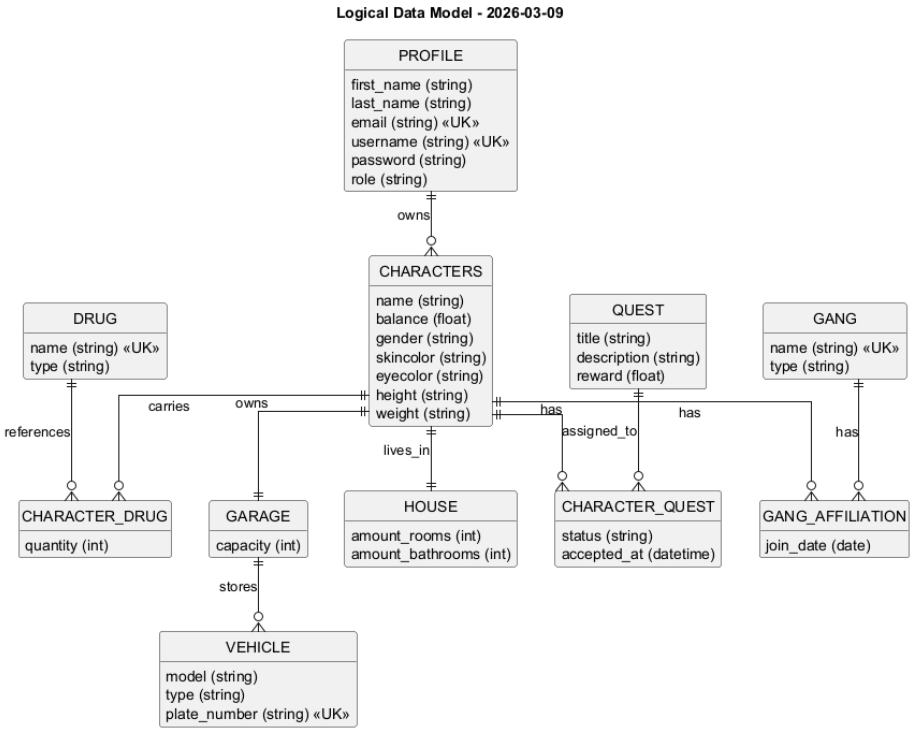


Figure 4: Logical data model. Authoritative source: 'logical-uml-09032026.puml'

Logical Diagram

Main Tables

- **Profiles** (first_name, last_name, email, username, password, role)
- **Characters** (name, balance, gender, skincolor, eyecolor, height, weight)
- **Houses** (amount_rooms, amount_bathrooms)
- **Garages** (capacity)
- **Vehicles** (model, type, plate_number)
- **Drugs** (name, type)
- **Character_Drug** (quantity)
- **Quests** (title, description, reward)
- **Character_Quest** (status, accepted_at)

- **Gangs** (name, type)
- **Gang__Affiliations** (join_date)

The logical layout keeps the model close to the gameplay language. `role`, `gender`, `skincolor`, and `eyecolor` are attributes on the owning entities, while relationship-heavy concepts such as gangs, quests, and drugs are normalized through link tables.

Logical Relationship Rules

- One `Profile` can own many `Characters`.
- Each `Character` must contain exactly one value for each required appearance attribute.
- `Character` and `House` are modeled as a mandatory one-to-one relationship.
- `Character` and `Garage` are modeled as a mandatory one-to-one relationship.
- `Garage` and `Vehicle` are modeled as one-to-many.
- `Character` and `Drug` are modeled through `Character_Drug`.
- `Character` and `Quest` are modeled through `Character_Quest`.
- `Character` and `Gang` are modeled through the junction table `Gang_Affiliations` (many-to-many).

In addition to these cardinalities, the logical model defines key strategy and uniqueness boundaries:

- `username` should be unique at profile level to guarantee a single login identity per player.

This structure keeps the data model clear and easier to maintain. Gameplay-facing attributes stay easy to read, while multi-valued relationships are still normalized where duplication would otherwise become a problem.

2.2.2. Normalization process

Normalization keeps data in one place and makes updates and queries more consistent. The goal is to eliminate redundancy and avoid insertion, update, and deletion anomalies that would otherwise appear in gameplay data.

First Normal Form (1NF) 1NF requires atomic attributes and no repeating groups in a single column. In this schema, all tables satisfy 1NF because each column holds one value per row. Multi-valued relationships such as character-gang, character-drug, and character-quest are modeled through separate tables.

Second Normal Form (2NF) 2NF requires that non-key attributes depend on the full key, not only part of it. This is especially relevant in relationship tables. `character_drug`, `character_quest`, and `gang_affiliations` satisfy

2NF because their descriptive attributes depend on the relationship pair, not on one side only.

Third Normal Form (3NF) 3NF removes transitive dependencies where non-key attributes depend on other non-key attributes. In the new model, `role`, `gender`, `skincolor`, and `eyecolor` are treated as direct domain attributes of their owning rows, so they are not transitive dependencies. Relationship metadata such as `join_date`, `status`, and `quantity` lives only in the appropriate junction tables.

The resulting 3NF design improves consistency and maintainability:

- Changes are localized to one table.
- Duplicate text values are minimized.
- Integrity constraints become easier to enforce.
- Queries stay clear and consistent as data grows.

The final logical schema therefore satisfies **3NF** and provides a reliable base for physical implementation.

2.3. Physical data model

Database setup and schema execution The physical database can be created directly from `sqls/schema.sql`. This makes onboarding simple: one script builds the tables, keys, and constraints used by the project.

Schema definition To make the database reproducible without access to the project host, a full schema script is included as `sqls/schema.sql`. The file contains hand-maintained DDL (Data Definition Language) for the final physical model used in the project.

The schema file acts as a portable definition of the physical data model. Anyone cloning the repository can create an identical empty database by running:

```
psql -U postgres -d bajls -f sqls/schema.sql
```

If your local database name is different, replace `bajls` with your own database name (for example `bajls_db`).

Minimum setup:

```
createdb bajls_db
psql -d bajls_db -f sqls/schema.sql
```

Clean reset (useful during development):

```
dropdb --if-exists bajls_db
createdb bajls_db
psql -d bajls_db -f sqls/schema.sql
```

If your PostgreSQL user is not the default user, specify it explicitly:

```
psql -U postgres -d bajls_db -f sqls/schema.sql
```

What the schema script creates The script creates the database structure in a logical order:

- Core identity tables first, such as `profiles`, `houses`, and `garages`.
- Main gameplay tables next, such as `characters`, `vehicles`, `drugs`, `quests`, and `gangs`.
- Relationship tables last: `character_drug`, `character_quest`, and `gang_affiliations`.

This order matters because foreign keys can only point to tables that already exist.

How constraints implement business rules Example from the same pattern used in `sqls/schema.sql`:

```
CREATE TABLE characters (
    id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    profile_id bigint NOT NULL REFERENCES profiles(id),
    house_id bigint NOT NULL UNIQUE REFERENCES houses(id),
    garage_id bigint NOT NULL UNIQUE REFERENCES garages(id)
);
```

This simple definition enforces important rules at database level:

- `PRIMARY KEY` gives each character a stable identity.
- `NOT NULL` makes profile, house, and garage assignment mandatory.
- `REFERENCES` prevents invalid IDs that do not exist in parent tables.
- `UNIQUE` on `house_id` enforces one house per character (one-to-one).

Many-to-many membership is handled with a junction table:

```
CREATE TABLE gang_affiliations (
    id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    character_id bigint NOT NULL REFERENCES characters(id),
    gang_id bigint NOT NULL REFERENCES gangs(id),
    join_date date NOT NULL,
    CONSTRAINT uk_gang_affiliations_character_gang UNIQUE
    ↪ (character_id, gang_id)
);
```

The unique constraint prevents duplicate memberships for the same character-gang pair while still allowing the table to use a simple surrogate primary key.

Quick verification after running the script After importing `sqls/schema.sql`, a developer can run short checks:

```
-- list created tables
\dt

-- inspect columns and constraints
\d characters
\d gang_affiliations
```

And a quick join test:

```sql
SELECT c.id, p.username, h.id AS house_id
FROM characters c
JOIN profiles p ON p.id = c.profile_id
JOIN houses h ON h.id = c.house_id
LIMIT 5;
```

If these queries run successfully, the schema is loaded and core relationships are working as expected.

The physical model is the concrete PostgreSQL implementation of the logical schema. At this stage, abstract relations are converted into actual database objects with column types, constraints, and execution characteristics. The implementation shown in pgAdmin reflects the final table structure and foreign-key network used by the project.

Physical Diagram

Implementation Details

- Primary keys are implemented as `bigint` identity columns.
- Foreign keys are created between dependent tables to enforce referential integrity.
- The `characters.house_id` column is unique, enforcing one house per character.
- The `characters.garage_id` column is unique, enforcing one garage per character.
- The `gang_affiliations` table stores many-to-many links between characters and gangs, including `join_date`.
- `character_drug` and `character_quest` store many-to-many links with relationship-specific attributes.
- Controlled values such as `role`, `gender`, `skincolor`, `eyecolor`, `vehicle.type`, `drug.type`, `gang.type`, and `character_quest.status` are enforced with SQL `CHECK` constraints and mirrored as Java enums.

The PostgreSQL schema is designed to enforce business rules at database level, not only in application code. Mandatory fields are protected with `NOT NULL`, key relationships are protected with foreign keys, and entity identity is protected with primary-key constraints. This reduces the risk of inconsistent data even if multiple services or scripts write to the same database.

For relationship-heavy queries, performance is important. Primary keys and foreign keys help the database connect tables efficiently for common operations such as loading profile characters, character attributes, and gang memberships. As data volume grows, this helps avoid scanning entire tables for routine gameplay queries.

Operationally, the physical design also supports maintainability:

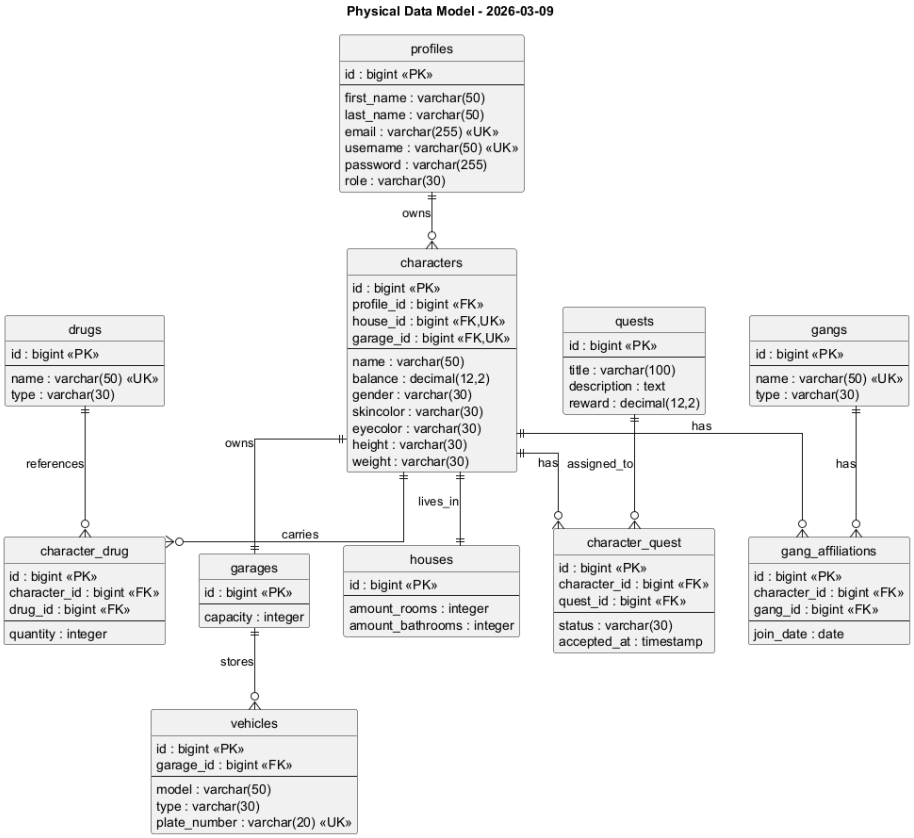


Figure 5: Physical data model. Authoritative source: ‘physical-uml-03092026.puml‘

- Clear naming conventions for tables and columns.
- Dedicated junction table for many-to-many associations.
- SQL `CHECK` constraints and enums for controlled value domains.
- Predictable join paths for reporting and administration tools.

Cardinality and Modality in the Physical Model At the physical level, cardinality and modality are enforced through foreign keys, uniqueness, and nullability:

- `profiles.role` is `NOT NULL`, so each profile must have exactly one role value.
- `characters.profile_id` is `NOT NULL`, so each character must belong to one profile, while a profile can still have zero or many characters.
- `characters.house_id` is both `NOT NULL` and `UNIQUE`, enforcing one

mandatory house per character and preventing multiple characters from referencing the same house.

- `characters.garage_id` is both `NOT NULL` and `UNIQUE`, enforcing one mandatory garage per character.
- Required character attributes such as `gender`, `skincolor`, `eyecolor`, `height`, and `weight` are stored directly on the `characters` row and protected with `NOT NULL`. The controlled domains for `gender`, `skincolor`, and `eyecolor` are further restricted with SQL `CHECK` constraints.
- `gang_affiliations` implements optional many-to-many membership: both sides can have zero or many links, while each link row must reference exactly one character and one gang.

2.3.1. Data types

The physical schema uses PostgreSQL types that match the actual table definitions in `sqls/schema.sql`:

- `bigint GENERATED BY DEFAULT AS IDENTITY` for all surrogate primary keys
- `bigint` for foreign keys such as `characters.profile_id` and `vehicles.garage_id`
- `varchar(50)` for most names and controlled text values, for example `profiles.first_name`, `characters.name`, and `gangs.name`
- `varchar(255)` for longer login-related text such as `email` and `password`
- `varchar(100)` for `quests.title`
- `text` for `quests.description`
- `numeric(12,2)` for monetary values such as `characters.balance` and `quests.reward`
- `integer` for count-like values such as room counts, garage capacity, and drug quantity
- `date` for `gang_affiliations.join_date`
- `timestamp` for `character_quest.accepted_at`

These choices balance simplicity and correctness. `bigint` identity keys provide stable row identity, `varchar` columns keep bounded text easy to validate, and `numeric(12,2)` is safer than floating-point types for game economy values. Temporal fields are split into `date` and `timestamp` depending on whether only a day or an exact acceptance time is required.

2.3.2. Primary and foreign keys

Primary keys use auto-generated `bigint` IDs (`GenerationType.IDENTITY`) in all main tables. This gives each row a simple and stable identifier.

Foreign keys encode the core relationships:

- `characters.profile_id` -> `profiles.id`
- `characters.house_id` -> `houses.id`
- `characters.garage_id` -> `garages.id`
- `vehicles.garage_id` -> `garages.id`
- `character_drug.character_id` -> `characters.id`
- `character_drug.drug_id` -> `drugs.id`
- `character_quest.character_id` -> `characters.id`
- `character_quest.quest_id` -> `quests.id`
- `gang_affiliations.character_id` -> `characters.id`
- `gang_affiliations.gang_id` -> `gangs.id`
- `characters.house_id` -> `houses.id` (unique)
- `characters.garage_id` -> `garages.id` (unique)

The many-to-many relation between characters and gangs is implemented with `gang_affiliations`, where the unique constraint on (`character_id`, `gang_id`)

prevents duplicate memberships for the same pair.

Why use `id` in junction tables? In a purely relational design, a junction table can use the two foreign keys as its primary key. That is a clean and classic solution.

In this project, we instead use a simple `id` primary key together with a `UNIQUE` constraint on the foreign-key pair. This was chosen because it makes the tables easier to work with in Java/Hibernate, while still preventing duplicate relationships.

Example:

In `gang_affiliations`, a row can look like this:

```
id = 12
character_id = 3
gang_id = 7
join_date = 2025-03-22
```

Here, `id` is the technical primary key, while `UNIQUE(character_id, gang_id)` ensures that character `3` cannot be added to gang `7` more than once.

Pros:

- **Using a separate `id`** makes CRUD operations and ORM mapping simpler.
- **Using `UNIQUE(character_id, gang_id)`** still protects the real business rule and prevents duplicate links.

Cons:

- The table gets one extra technical key that is not part of the real business relationship.
- A pure relational design would be slightly cleaner with only the two foreign keys as the primary key.

For this project, the trade-off is acceptable because the code becomes easier to read and maintain.

2.3.3. Indexes

Indexes are used where lookup speed matters for common application queries. The relational schema includes indexes on profile identity fields such as `email` and `username`, because login and profile lookup depend on these values. Primary keys also create index-backed access paths automatically. This supports fast lookup for endpoints such as `GET /api/characters/{id}` and for joins through foreign keys.

2.3.4. Constraints and referential integrity

The schema enforces integrity through a combination of column constraints and relationship constraints:

- `NOT NULL` is used on mandatory attributes (for example profile names, credentials, role values, character references, and `join_date`).
- `UNIQUE` constraints prevent duplicates on identity-like values: `profiles.email`, `profiles.username`, `gangs.name`, `characters.house_id`, `characters.garage_id`, and each junction-table character-pair combination.
- Foreign-key constraints ensure references remain valid across table boundaries. Examples include `fk_characters_profile`, `fk_characters_house`, `fk_characters_garage`, `fk_character_drug_character`, `fk_character_quest_quest`, `fk_gang_aff_char`, and `fk_gang_aff_gang`.

Controlled value domains are protected directly in SQL with `CHECK` constraints instead of separate lookup tables. Examples are:

- `chk_profiles_role` for `USER` / `ADMIN`
- `chk_characters_gender`
- `chk_characters_skincolor`
- `chk_characters_eyecolor`
- `chk_vehicles_type`
- `chk_drugs_type`
- `chk_character_quest_status`
- `chk_gangs_type`

In JPA/Hibernate, required references are marked as mandatory. This matches the database rules and helps prevent missing links, unclear ownership, and duplicate relationships.

2.4. Stored objects - views, triggers, events

Views are virtual tables defined by SQL queries. Instead of storing data physically, a view presents data from one or more underlying tables through a predefined query. This allows users and applications to access filtered, structured, or combined data without interacting directly with the base tables.

A view behaves like a regular table in queries. Users can perform `SELECT` operations on a view, while the database executes the underlying query dynamically and returns the result set. Because the data is not stored separately, a view always reflects the current state of the underlying tables.

Views are useful when data should be presented in a controlled or simplified form. In this RPG project, they are used to expose selected character information, player summaries, and administrative overviews without requiring direct access to the full database structure.

This section covers read-oriented objects (`views`), write-time validation (`triggers`), and time-based automation (`events` implemented through `pg_cron`).

2.4.1. Views

Simplification of complex queries: A view can encapsulate joins and filters that would otherwise require long and complex SQL statements. This makes application queries easier to write and maintain.

Abstraction from physical structure: Views create a logical layer between the application and the database tables. If table structures change, only the view definition needs to be updated, while application queries can remain unchanged.

Security and access control: Views can restrict access to sensitive data by exposing only selected columns or rows. For example, administrative or private fields such as passwords or internal identifiers can be hidden.

Low storage overhead: Standard views do not store data physically, since they generate results dynamically from the base tables.

Limitations of Views **Performance overhead:** Since the underlying query is executed each time the view is accessed, complex views can reduce performance, especially when based on multiple joins or large tables.

Dependency on base tables: A view depends on the structure of its underlying tables. If referenced columns or tables are changed or removed, the view may become invalid.

Update Limitations: Not all views are updatable, particularly those involving joins, grouping, or complex calculations.

How Views Will Be Used in the RPG Project In the RPG system, many features require data from multiple related tables such as profiles, characters, and gangs.

Simplify complex queries:
Instead of writing long SQL statements with multiple joins, the application can query a single view.

Provide abstraction: Views create a logical layer between the application and the physical database. If the table structure changes, only the view definition needs to be updated, reducing the impact on the application code.

Improve security: Views can hide sensitive information such as passwords or internal identifiers and expose only the data needed by the application or administrative tools.

Support administration and reporting: Views can present summarized information about players, characters, houses, and gang memberships in a clear and structured format. This approach improves maintainability and ensures controlled access to the game data.

Character Overview View The current version of `v_character_overview` combines account, character, housing, garage, and gang information into one read-friendly result:

```
CREATE OR REPLACE VIEW v_character_overview AS
SELECT
    p.username,
    c.name AS character_name,
    c.balance,
    h.amount_rooms,
    h.amount_bathrooms,
    g.capacity AS garage_capacity,
    COALESCE(STRING_AGG(DISTINCT ga2.name, ', '), 'No gang')
     ↪  AS affiliations
FROM characters c
JOIN profiles p ON p.id = c.profile_id
JOIN houses h ON h.id = c.house_id
JOIN garages g ON g.id = c.garage_id
LEFT JOIN gang_affiliations gaf ON gaf.character_id = c.id
LEFT JOIN gangs ga2 ON ga2.id = gaf.gang_id
GROUP BY p.username, c.name, c.balance, h.amount_rooms,
 ↪  h.amount_bathrooms, g.capacity;
```

This view is useful because it collects the most common “profile page” data into one query result: who owns the character, where the character lives, how large the garage is, and which gangs the character belongs to.

Characters to Gangs (optional many-to-many relationship)

Gang membership is still optional and is still modeled through the junction table `gang_affiliations`. A `LEFT JOIN` is therefore used so that characters without gang membership still appear in the result.

Aggregation and Grouping

`STRING_AGG` is used to combine multiple gang names into one readable column. Because a character can have many gang memberships, grouping is required to collapse those rows into one output row per character.

Result

When querying the view:

```
SELECT * FROM v_character_overview;
```

	username character varying (20)	name character varying (20)	balance real	affiliations text
1	aandersen	AdminAsta	5000	Echo Unit
2	enielsen	RuneEmma	3890.75	Solar Pact
3	ijensen	Novalda	4150	Spineless
4	lpedersen	VoltLucas	980.25	Crimson Tide
5	mihansen	ShadowMia	2450.5	Neon Foxes, Night Owls
6	mihansen	ShadowMiaAlt	1500	Spineless
7	nlarsen	SteelNoah	1320	Iron Wolves
8	omadsen	HexOliver	2100.4	Frost Syndicate
9	skristensen	BlazeSofia	1750.1	Spineless
10	vsorensen	ModVictor	4600.6	Spineless
11	wolsen	FrostWill	2999.99	Spineless

Figure 6: v_character_overview

This structure provides a clear and compact overview suitable for application use, administration, and reporting.

Gang Overview View

Gang Overview View

The `v_gang_overview` view provides a summarized overview of each gang, its type, and its members.

```sql
CREATE OR REPLACE VIEW v_gang_overview AS
SELECT
    gangs.name AS gang_name,
    COALESCE(STRING_AGG(characters.name, ', '), 'No members')
     ↪  AS members
FROM gangs
LEFT JOIN gang_affiliations ON gang_affiliations.gang_id =
 ↪  gangs.id
LEFT JOIN characters ON characters.id =
 ↪  gang_affiliations.character_id
GROUP BY gangs.name;
```

Result

Result

When querying the view:

```sql
SELECT * FROM v_gang_overview;
```

	gang_name character varying (20)	members text
1	Echo Unit	AdminAsta
2	Solar Pact	RuneEmma
3	Crimson Tide	VoltLucas
4	Sand Vipers	No members
5	Frost Syndicate	HexOliver
6	Iron Wolves	SteelNoah
7	Stone Guard	No members
8	Night Owls	ShadowMia
9	Sky Raiders	No members
10	Neon Foxes	ShadowMia

Figure 7: v_gang_overview

The view combines data from the `gangs`, `gang_affiliations`, and `characters` tables to present gang information in a simplified format.

Character Appearance Overview View

The `v_character_appearance` view provides a structured overview of each character's physical attributes.

```
CREATE OR REPLACE VIEW v_character_appearance AS
SELECT
    characters.name AS character_name,
    characters.balance AS balance,
    characters.gender AS gender,
    characters.weight AS weight,
    characters.height AS height,
    characters.eyecolor AS eyecolor,
    characters.skincolor AS skincolor
FROM characters
```

In the updated schema these appearance values are stored directly in the `characters` table. The view is therefore simpler than before and no longer depends on lookup joins.

Result

When querying the view:

```
SELECT * FROM v_character_appearance;
```

	character_name character varying (20)	balance real	gender character varying (20)	weight character varying (20)	height character varying (20)	eye_color character varying (20)	skin_color character varying (20)
1	ShadowMia	2450.5	FEMALE	AVERAGE	AVERAGE	BLUE	LIGHT
2	SteelNoah	1320	MALE	HEAVY	TALL	BROWN	MEDIUM
3	RuneEmma	3890.75	FEMALE	LIGHT	AVERAGE	GREEN	MEDIUM
4	VoltLucas	980.25	MALE	AVERAGE	SHORT	GRAY	DARK
5	Novalda	4150	FEMALE	AVERAGE	TALL	BROWN	LIGHT
6	HexOliver	2100.4	MALE	HEAVY	AVERAGE	BLUE	MEDIUM
7	BlazeSofia	1750.1	FEMALE	LIGHT	SHORT	GREEN	DARK
8	FrostWill	2999.99	MALE	AVERAGE	AVERAGE	GRAY	LIGHT
9	AdminAsta	5000	OTHER	AVERAGE	TALL	BROWN	MEDIUM
10	ModVictor	4600.6	MALE	HEAVY	AVERAGE	BLUE	DARK
11	ShadowMiaAlt	1500	FEMALE	AVERAGE	AVERAGE	BLUE	LIGHT

Figure 8: v_character_overview

The view simplifies queries by collecting all appearance-related information in a single logical structure.

2.4.2. Triggers

A trigger is a piece of SQL code that runs automatically in response to certain events on a table. A trigger can be set to execute before or after an `INSERT`,

`UPDATE`, or `DELETE` operation. Triggers are commonly used to enforce complex business rules, maintain data integrity, or perform automatic updates based on changes in the database.

Purpose of trigger in this project

In this project, a trigger is linked to the characters table. The purpose of the trigger is to automatically react whenever a new character is created.

Character creation is a core operation in our RPG game. Each time a new character is inserted into the database, the system should generate a message indicating that the character has been successfully created. Rather than implementing this behavior solely in application code, it is defined directly in PostgreSQL. This guarantees that the behavior is executed consistently, even if data insertion occurs from administrative scripts, test environments, or other services.

Trigger Implementation

The trigger is defined as an AFTER INSERT trigger on the characters table and executes once per inserted row (FOR EACH ROW). When a new character is inserted, PostgreSQL invokes the trigger function and exposes the inserted row through the special record variable NEW.

The trigger function uses RAISE NOTICE to send a notice containing the newly created character's name. Because the trigger runs after the insert operation, referential integrity is guaranteed at the time the message is produced.

The trigger looks like this:

```
CREATE OR REPLACE FUNCTION fn_character_notice()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  RAISE NOTICE 'Welcome! Your new character created: name=%',
 ↪  NEW.name;
RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_character_notice ON characters;

CREATE TRIGGER trg_character_notice
    AFTER INSERT ON characters
    FOR EACH ROW
    EXECUTE FUNCTION fn_character_notice();
```

The message is not stored in a database table. Instead, it is sent as a notice to the client application that performed the insert. This allows real-time feedback without modifying the data model or adding extra tables for logging.

Design considerations

Design considerations

Using a trigger for this purpose has both advantages and disadvantages:

- **Advantages**:
  - Ensures consistent behavior regardless of how data is inserted.
  - Centralizes the logic in the database, reducing reliance on application code.
  - Provides immediate feedback to users or administrators when characters are created.
- **Disadvantages**:
  - Reduces transparency of system control flow.
  - Debugging may require inspecting database-level logic.
  - The message is not stored for later retrieval.

Given the limited scope of the project, this approach provides a clear demonstration of automated database behavior without introducing additional complexity.

2.4.3. Events

Introduction In many relational database courses, "events" usually means scheduled database tasks that run automatically at fixed times. PostgreSQL does not provide a built-in `CREATE EVENT` feature like MySQL. For this reason, this project implements scheduled behavior using a cron-based approach instead.

The implementation is placed in `sqls/daily_loyalty_bonus.sql`.

Why cron jobs in PostgreSQL Because PostgreSQL has no native event scheduler syntax, we use the `pg_cron` extension to execute SQL commands on a defined schedule. This gives us event-like behavior while staying inside PostgreSQL.

In this project, the scheduled task runs every day at **00:01** and updates each character's balance according to current activity:

- all characters receive a base bonus
- extra bonus is added for gang memberships
- extra bonus is added for active quest assignments

Cron job file walkthrough The full implementation is in `sqls/daily_loyalty_bonus.sql` and is structured in three parts:

1. Enable extension:

```
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

This ensures cron scheduling is available in the current database.

2. Replace existing job with same name:

```
DO $$
DECLARE
    v_job_id integer;
BEGIN
    SELECT jobid
    INTO v_job_id
    FROM cron.job
    WHERE jobname = 'daily_loyalty_bonus';

    IF v_job_id IS NOT NULL THEN
        PERFORM cron.unschedule(v_job_id);
    END IF;
END $$;
```

This prevents duplicate schedules if the script is executed multiple times.

3. Schedule daily payout at `00:01`:

```
SELECT cron.schedule(
    'daily_loyalty_bonus',
    '1 0 * * *',
    $cron$
    UPDATE characters c
    SET balance = c.balance + COALESCE(b.bonus, 100)
    FROM (
        SELECT
            c2.id AS character_id,
            (100 + COUNT(DISTINCT ga.gang_id) * 25 +
 ↪  COUNT(DISTINCT cq.quest_id) * 10)::numeric(12,2) AS bonus
        FROM characters c2
        LEFT JOIN gang_affiliations ga ON ga.character_id =
         ↪  c2.id
        LEFT JOIN character_quest cq ON cq.character_id =
         ↪  c2.id
        GROUP BY c2.id
    ) b
    WHERE c.id = b.character_id;
    $cron$
);
```

How this works:

- `'1 0 * * *'` means every day at 00:01.
- `LEFT JOIN` keeps all characters in scope, even without gangs or quests.
- each gang adds `25` bonus.
- each quest relation adds `10` bonus.
- `COALESCE(b.bonus, 100)` still guarantees a fallback bonus.

Summary PostgreSQL does not have native event objects, so scheduled automation is implemented using cron jobs. In this project, the daily loyalty payout is implemented as a reusable SQL script in `sqls/daily_loyalty_bonus.sql`, providing predictable and fully database-side automation.

2.4.4. Realistic seed data

Introduction

The project includes a dedicated seed script, `sqls/seed.sql`, that creates a realistic baseline dataset for demos, testing, and validation. Instead of random values, the script inserts coherent player profiles, characters, houses, garages, vehicles, quests, drugs, and gang memberships, so queries return believable results and relationship rules can be verified in practice. The current dataset is intentionally large and connected, so it is also useful for MongoDB document inspection and Neo4j graph visualization.

Script Structure

The script starts by opening a transaction and resetting all relevant tables. This makes each run deterministic and easy to repeat.

In this context, *deterministic* means that the script produces the same data state every time it is executed on the same schema.

```
BEGIN;
TRUNCATE TABLE
    gang_affiliations,
    character_quest,
    quests,
    character_drug,
    drugs,
    vehicles,
    characters,
    garages,
    profiles,
    gangs,
    houses
RESTART IDENTITY CASCADE;
```

`TRUNCATE` removes all rows from the listed tables very quickly. `RESTART IDENTITY` resets auto-increment IDs back to their starting values, and `CASCADE` ensures dependent tables are also cleared safely.

After reset, the script inserts profiles, houses, garages, characters, vehicles, drugs, quests, gangs, and finally the relationship rows. The current version seeds:

- 82 profiles
- 82 houses
- 82 garages
- 82 characters
- 123 vehicles

- 40 drugs
- 48 quests
- 40 gangs
- 266 `character_drug` rows
- 246 `character_quest` rows
- 291 `gang_affiliations` rows

Character, house, and garage seeding

The new model no longer needs schema-detection logic. The seeding order follows the actual foreign keys directly:

1. `profiles`
2. `houses`
3. `garages`
4. `characters`
5. `vehicles`
6. relationship tables

Gang membership data

Gang membership is optional in the domain model, but the seed is deliberately denser than the minimum model requires. Most characters receive multiple rows in `gang_affiliations`, and several gangs are used as hub gangs with many members. This makes the graph representation much more informative, because a Neo4j visualization now shows clusters and shared memberships instead of mostly isolated pairs.

```
INSERT INTO gang_affiliations (character_id, gang_id,
 ↪  join_date) VALUES
    (1, 1, '2026-02-01'),
    (1, 10, '2026-03-05'),
    (1, 22, '2026-04-09'),
    (5, 2, '2026-04-04'),
    (41, 2, '2026-01-19');
```

Finally, all inserts are persisted in one atomic commit.

```
COMMIT;
```

2.4.5. Stored functions and procedures

Introduction

Stored functions and procedures are database objects that encapsulate SQL code for reuse. Functions return values and can be used in queries, while procedures perform actions and are not required to return a value. Both reduce redundancy and centralize business logic within the database.

Stored Functions

A stored function is a programmable database object that encapsulates reusable SQL logic and always returns a single value. It is created with the `CREATE FUNCTION` statement and stored in the database for repeated use. Functions are well suited to calculations that need to be reused across multiple queries. All functions used in this project are located in `sqls/functions`.

What defines a stored function:

- It's created by the user
- It's stored inside the database
- Belongs to a schema
- Accepts input parameters
- Must return exactly one value
- Can be used inside SQL statements (e.g., in SELECT, WHERE, ORDER BY)

Example of Stored Function Usage in the Project The project currently includes one concrete stored function in `sqls/functions/get_wealth.sql`: `get_wealth_status(p_balance numeric)`. It returns a text category for a character's balance and is imported as part of database setup with:

```
psql -d bajls_db -f sqls/functions/get_wealth.sql
```

The following snippet shows the actual implementation used in the project:

```
CREATE OR REPLACE FUNCTION get_wealth_status(p_balance
 ↪  numeric)
RETURNS text
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_balance < 1000 THEN
```

```
        RETURN 'poor';
    ELSIF p_balance < 3000 THEN
        RETURN 'middleclass';
    ELSE
        RETURN 'rich';
    END IF;
END;
$$;
```

1. Input: The function expects a numeric input called `p_balance`, which represents a character's balance.
2. Process: The function uses `IF / ELSIF / ELSE` logic to classify the balance: `poor` for values below `1000`, `middleclass` for values below `3000`, and `rich` otherwise.
3. Output: The result is a text value representing the character's wealth status.

Stored procedures

Stored procedures group SQL statements and business logic into a single reusable unit that runs inside the database. Unlike a stored function, a procedure does not have to return a value and is typically used to perform actions such as inserting, updating, deleting, or coordinating multiple SQL operations in sequence. This helps enforce consistent business rules across the system. All procedures used in this project are located in `sqls/procedures`.

The following example shows a procedure that handles the required operations for creating a character together with both a house and a garage.

The full code is located in `sqls/procedures/create_character_with_house.sql` and can be installed with:

```
psql -d bajls_db -f
 ↪  sqls/procedures/create_character_with_house.sql
```

Procedure workflow:

1. Input parameters:
   - The procedure accepts values for profile, appearance, house, and garage data.
2. House creation:
   - A new row is inserted into `houses`.
3. Garage creation:
   - A new row is inserted into `garages`.
4. Character creation:

- The character is inserted with both `house_id` and `garage_id`.

2.5. Transactions

The relational application uses transactions in the DAO layer for create, update, and delete operations. Each write operation starts a transaction, performs the database operation, flushes changes where needed, and commits if no exception occurs. If a runtime exception is raised, the transaction is rolled back before the exception continues to the HTTP layer.

This structure ensures that partial writes are not persisted. For example, if an entity update fails because a foreign key is invalid or a constraint is violated, the operation is rolled back and the database remains consistent. PostgreSQL also protects multi-table operations inside stored procedures, such as creating a character with house and garage data, by treating the procedure call as part of the surrounding transaction.

The migration application also follows a clear transaction boundary per target database operation. PostgreSQL is read as the source snapshot, then MongoDB or Neo4j is cleared and rewritten from that snapshot. This keeps the migration process deterministic during local development and testing.

2.6. Auditing

The original project included an audit-log design, but the final simplified implementation deliberately removes the audit feature. This means there is no `audit_log` table, no audit triggers, and no audit API route in the delivered code.

The design decision was made because the project became unnecessarily complex for the current learning goals. Removing audit logging made the domain model, repositories, and routes easier to understand and reduced the amount of hidden behavior during CRUD operations. If audit logging were reintroduced later, it should be implemented as a focused PostgreSQL trigger-based feature with a separate `audit_log` table and a clear administrator-only read endpoint.

2.7. Security

2.7.1. Explanation of users and privileges

The system uses only two roles: `USER` and `ADMIN`.

- `USER`: regular player role with access to normal gameplay features (profile and character usage).
- `ADMIN`: administrative role with extended permissions for management and moderation tasks.

Each profile has exactly one role, and a profile cannot have both roles at the

same time. This keeps access control simple and aligned with the project requirements.

2.7.2. SQL Injection

SQL injection happens when untrusted user input is concatenated directly into SQL strings and executed as code. The project avoids this by using JPA parameter binding, Criteria API queries, and fixed JPQL projection queries instead of building SQL from raw request strings.

Authentication queries use named parameters such as `:username` and `:password`, so user input is sent as values rather than executable SQL. CRUD lookup methods use numeric path parameters that are parsed to `Long` before database access. This prevents classic string-based SQL injection in the implemented data layer.

2.8. Description of the CRUD application for RDBMS

This section documents the implemented backend structure in the Java application after the simplification of the route and persistence-facing layers. The goal was to keep the code straightforward and aligned with the new model instead of preserving the earlier, more generic design.

Current request flow in the relational implementation:

- `Route -> CrudController -> DAO -> DTO -> JSON response`
- `Auth route -> AuthService -> ProfileDao -> LoginResponseDTO`

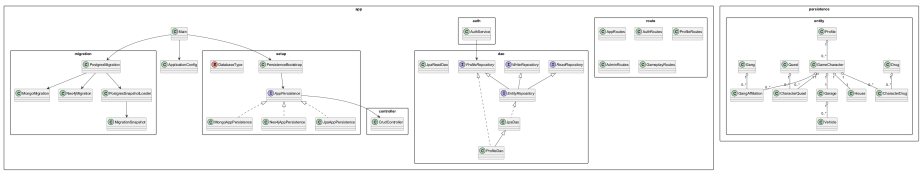


Figure 9: Backend module/class diagram

ApplicationConfig (Javalin bootstrap) `ApplicationConfig` defines the API baseline for all endpoints using Javalin[7]:

- default content type: JSON
- context path: `/api`
- CORS enabled (development-friendly)

Code example:

[7] Javalin Documentation: https://javalin.io/documentation

```
app = Javalin.create(config -> {
    config.http.defaultContentType = "application/json";
    config.routing.contextPath = "/api";
    config.plugins.enableCors(cors ->
 ↪  cors.add(CorsPluginConfig::anyHost));
});
```

This ensures the whole API runs with a consistent base path and response format.

Auth (login/register/logout + role-based authorization) Authentication is intentionally simple:

- `POST /api/auth/login`
- `POST /api/auth/register`
- `POST /api/auth/logout`

Protected routes use Basic Authentication in the `Authorization` header. `AuthService` reads the credentials, authenticates against `ProfileDao`, and then applies either:

- authenticated-user access
- `ADMIN` access
- profile-owner-or-admin access for `/profiles/{id}`

Because the application uses stateless Basic Authentication, logout does not invalidate a server-side session. Instead, `POST /api/auth/logout` acts as an explicit client-facing endpoint that confirms the user should remove the `Authorization` header. This keeps the flow simple while still covering login/logout behavior in the API contract.

This is simpler than the previous session/token design and is easier to trace in the code.

init.sql (database users and privileges) In addition to application-level roles, database-level privileges are defined in `db/init.sql` using least-privilege principles.

Implemented roles:

- `bajls_readonly`
- `bajls_readwrite`
- `bajls_app_user` (login, readonly membership)
- `bajls_app_admin` (login, readwrite membership)

Code example:

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO bajls_readonly;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA
 ↪  public TO bajls_readwrite;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO
 ↪  bajls_readwrite;
```

This creates a clear separation between read-only and write-capable database access.

DAO layer The DAO layer is implemented in `src/main/java/app/dao` with a concrete class structure:

- `JpaDao<T>` as the concrete class for entity CRUD
- `JpaReadDao<T>` for DTO projections
- entity-specific DAOs such as `ProfileDao`, `GameCharacterDao`, `GarageDao`, `VehicleDao`, `DrugDao`, and `QuestDao`

Code example (generic save with transaction handling):

```
tx.begin();
em.persist(entity);
tx.commit();
```

Code example (safe `findAll` using Criteria API):

```
CriteriaQuery<T> criteria =
 ↪  em.getCriteriaBuilder().createQuery(entityClass);
criteria.from(entityClass);
return em.createQuery(criteria).getResultList();
```

Using Criteria API here avoids string-concatenated dynamic queries and improves safety/readability for entity CRUD. DTO projection queries are kept explicit in `JpaReadDao`.

Controller layer The controller layer is simplified to one reusable class:

- `CrudController<R, W>`

This controller handles:

- `getAll`
- `getById`
- `create`
- `update`
- `delete`

This makes route classes shorter and avoids duplicating the same HTTP parsing logic across resources.

DTO layer The DTO layer is implemented in `src/main/java/app/dto` using simple Java classes instead of records. Each important resource has a DTO, for example:

- `ProfileDTO`
- `GameCharacterDTO`
- `GangAffiliationDTO`
- `GarageDTO`
- `VehicleDTO`
- `DrugDTO`
- `QuestDTO`
- `CharacterQuestDTO`

DTOs are intentionally not required to be 1:1 with the database model. For example, `ProfileDTO` does not expose `password`, even though the field exists in the `profiles` table/entity.

Code example:

```
public class ProfileDTO {
    private Long id;
    private String firstName;
    private String lastName;
    private String email;
    private String username;
    private ProfileRole role;
}
```

DTOs decouple API payloads from JPA entities and avoid serialization problems with lazy-loaded relationships.

Main startup integration `Main` wires reset, migration, persistence, and HTTP startup:

```
PostgresReset.resetAndSeed(false);
PostgresMigration.migrateAll(false);

AppPersistence postgresPersistence = PersistenceBootstrap.
 ↪ createPersistence(DatabaseType.POSTGRES,
 ↪ false);
```

```
AppPersistence mongoPersistence = PersistenceBootstrap.
 ↪  createPersistence(DatabaseType.MONGODB,
 ↪  false);
AppPersistence neo4jPersistence =
 ↪  PersistenceBootstrap.createPersistence(DatabaseType.NEO4J,
 ↪  false);

new ApplicationConfig().start(7072,
 ↪  AppRoutes.build(postgresPersistence));
new ApplicationConfig().start(7073,
 ↪  AppRoutes.build(mongoPersistence));
new ApplicationConfig().start(7074,
 ↪  AppRoutes.build(neo4jPersistence));
```

This startup flow ensures that PostgreSQL is always rebuilt from a clean seed before MongoDB and Neo4j are repopulated from the same source snapshot. It also makes the three API implementations available at fixed ports in one run: PostgreSQL on `7072`, MongoDB on `7073`, and Neo4j on `7074`.

2.8.1. Required delivery artifacts

This subsection provides the required artifacts for security scripts, source code access, and installation in a test environment.

SQL scripts for creation of users and privileges The SQL script used to create database users and assign privileges is included in:

- `db/init.sql`

Example excerpt:

```
CREATE ROLE bajls_readonly NOLOGIN;
CREATE ROLE bajls_readwrite NOLOGIN;
CREATE ROLE bajls_app_user LOGIN PASSWORD 'change_me_user';
CREATE ROLE bajls_app_admin LOGIN PASSWORD 'change_me_admin';

GRANT bajls_readonly TO bajls_app_user;
GRANT bajls_readwrite TO bajls_app_admin;
```

This script implements least-privilege access control at database level.

Source code of the CRUD application (public repository) The full CRUD application source code is available at[8]:

[8] Project source repository: https://github.com/AhmadAlkaseb/Bajls

- https://github.com/AhmadAlkaseb/Bajls

Brief installation procedure (test environment) The following procedure sets up the system with full operational capabilities in a local test environment.

1. Clone repository:

```
git clone https://github.com/AhmadAlkaseb/Bajls.git
cd Bajls
```

2. Start database services with Docker:

```
docker compose up -d
```

3. Create the empty database if needed:

```
createdb bajls_db
```

4. Import the relational database objects in this order:

```
psql -d bajls_db -f db/init.sql
psql -d bajls_db -f sqls/schema.sql
psql -d bajls_db -f sqls/views.sql
psql -d bajls_db -f sqls/trigger.sql
psql -d bajls_db -f sqls/functions/get_wealth.sql
psql -d bajls_db -f
 ↪ sqls/procedures/create_character_with_house.sql
psql -d bajls_db -f sqls/daily_loyalty_bonus.sql
psql -d bajls_db -f sqls/seed.sql
```

This sequence creates users/privileges, tables/constraints, read models, trigger logic, reusable database code, scheduled behavior, and finally realistic test data.

In the current startup flow, PostgreSQL will be reset and reseeded again automatically when `app.Main` starts.

5. Set application environment variables (example values):

```
DB_URL=jdbc:postgresql://localhost:5432/bajls
DB_USER=postgres
DB_PASSWORD=postgres
```

6. Build and run the application:

```
mvn clean compile
mvn exec:java -Dexec.mainClass=app.Main
```

This starts all three API variants at the same time:

- PostgreSQL API: `http://localhost:7072/api`
- MongoDB API: `http://localhost:7073/api`
- Neo4j API: `http://localhost:7074/api`

7. Validate API availability:

```
POST http://localhost:7072/api/auth/login
```

8. Authenticate and test secured CRUD endpoints:

```
POST http://localhost:7072/api/auth/register
GET  http://localhost:7072/api/characters
```

Use Basic Authentication in the `Authorization` header for secured routes.

3. Document database

3.1. Intro to document databases

A document database stores data as JSON-like documents instead of rows in normalized tables. This model is useful when the application frequently reads complete aggregates (for example one profile with all characters, houses, and gang memberships) in a single request.

In this project, MongoDB[9] is used to model player-centric aggregates. The core idea is to keep closely related data together through embedding, while still allowing selected collections to remain separate for administrative and lookup use cases.

3.2. Database design - graphical form (collections and embeddings)

The MongoDB design source is located in `report/images/mongodb/design.json`.

Graphical structure (collection-level overview):

```
profiles (collection)
`-- profile document
    |-- _id
    |-- profile_id
    |-- email
    |-- first_name
    |-- last_name
    |-- username
    |-- password
    |-- role
    `-- characters [array]
        `-- character document
            |-- _id
            |-- name
            |-- balance
            |-- gender
            |-- skincolor
            |-- eyecolor
            |-- height
            |-- weight
            |-- house {embedded object}
            |   |-- _id
            |   |-- amount_rooms
            |   `-- amount_bathrooms
            |-- garage {embedded object}
```

[9] MongoDB Documentation: https://www.mongodb.com/docs/

```
            |   |-- _id
            |   |-- capacity
            |   `-- vehicles [array]
            |       `-- vehicle document
            |           |-- _id
            |           |-- model
            |           |-- type
            |           `-- plate_number
            |-- character_drugs [array]
            |   `-- relation document
            |       |-- drug_id
            |       `-- quantity
            |-- character_quests [array]
            |   `-- relation document
            |       |-- quest_id
            |       |-- status
            |       `-- accepted_at
            `-- gang_memberships [array]
                `-- relation document
                    |-- gang_id
                    `-- join_date

drugs (collection)
`-- drug document
    |-- _id
    |-- name
    `-- type

quests (collection)
`-- quest document
    |-- _id
    |-- title
    |-- description
    `-- reward

gangs (collection)
`-- gang document
    |-- _id
    |-- name
    `-- type
```

This design gives a profile-centric aggregate in `profiles`, while keeping `drugs`, `quests`, and `gangs` as separate shared collections.

3.3. Features: indexes, transactions, PKs, constraints, stored objects

Indexes

Typical MongoDB indexes for this design:

- unique index on `profiles.username`
- unique index on `profiles.email`
- index on `characters._id` or `characters.name` inside embedded array queries, if those access patterns are used
- index on `gangs.name`

Example:

```
db.profiles.createIndex({ username: 1 }, { unique: true });
db.profiles.createIndex({ email: 1 }, { unique: true });
db.gangs.createIndex({ name: 1 }, { unique: true });
```

Transactions

MongoDB supports multi-document ACID transactions (replica set / sharded cluster). In this project they are relevant when one logical operation updates both `profiles` and `gangs` collections.

Primary keys

MongoDB automatically assigns `_id` as primary key for each document. Domain IDs (`profile_id`, `character_id`, `gang_id`) are additional business identifiers used by the application.

Constraints and validation

MongoDB does not enforce relational foreign keys. Integrity is achieved through:

- schema validation rules (`$jsonSchema`)
- unique indexes
- application-level checks in the service/DAO layer

Stored objects and replacement strategy

MongoDB does not use SQL-style stored procedures/triggers/views in the same way as PostgreSQL. Equivalent behavior is implemented through:

- aggregation pipelines (view-like read models)
- application services (business logic orchestration)
- scheduled jobs in application/infrastructure layer for timed behavior

3.4. CRUD application for the document database

The HTTP/API structure follows the same style as the relational application (Javalin routes, auth guards, DTO-based payloads). The main difference is the data layer implementation:

- RDBMS version: JPA entities and DAOs
- MongoDB version: collection/document operations with embedded updates

Because the contract can remain the same, most differences are isolated to persistence adapters and query/update logic.

Example difference in write behavior:

- RDBMS: insert into `characters`, then link to `houses` and `gang_affiliations`.
- MongoDB: update one profile aggregate document with embedded character, nested house/garage/vehicle data, and relation arrays such as `gang_memberships`.

4. Graph database

4.1. Intro to graph databases

A graph database models data as nodes and relationships. It is strong for domains where connections are first-class and traversal queries are central (for example social links, memberships, shortest paths, and network analysis).

In this project, Neo4j[10] represents the same RPG domain using labeled nodes (`Profile`, `Character`, `Gang`, etc.) and typed relationships (`OWNS_CHARACTER`, `MEMBER_OF`, etc.).

4.2. Database design - graphical form (model, not data screenshot)

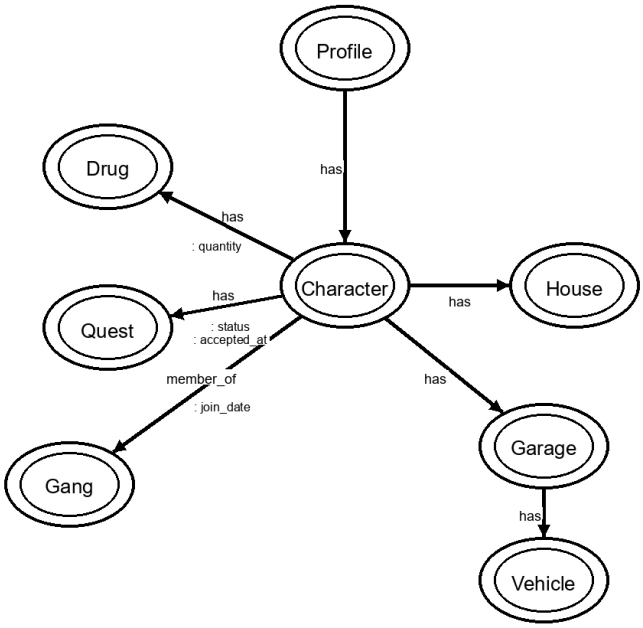


Figure 10: Neo4j database model (nodes and relationships)

The model keeps relationship semantics explicit. In particular, membership between character and gang is represented by `[:MEMBER_OF {join_date}]`, where `join_date` is stored on the relationship itself.

[10] Neo4j Documentation: https://neo4j.com/docs/

4.3. Features: indexes, transactions, PKs, constraints, stored objects

Indexes and constraints

Neo4j uses schema indexes and constraints to enforce integrity and speed up lookups.

Examples:

```
CREATE CONSTRAINT profile_id_unique IF NOT EXISTS
FOR (p:Profile) REQUIRE p.id IS UNIQUE;

CREATE CONSTRAINT gang_id_unique IF NOT EXISTS
FOR (g:Gang) REQUIRE g.id IS UNIQUE;

CREATE INDEX character_name_idx IF NOT EXISTS
FOR (c:Character) ON (c.name);
```

Transactions

Neo4j operations execute transactionally. Multi-step graph updates (for example creating a character node and connecting it to profile, house, and attributes) can be committed atomically.

Primary keys

Neo4j has an internal node id, but domain-level IDs should be managed as properties (for example `Profile.id`) with uniqueness constraints. This is the practical equivalent of a primary-key strategy in graph models. In other words, the project does not rely on Neo4j's internal node ids as business identifiers. Instead, stable application-defined ids are stored as node properties and protected with uniqueness constraints.

Constraints

Relational foreign keys are replaced by controlled relationship creation/traversal patterns and constraints on node identity. This means the graph model enforces uniqueness for important node ids, while valid links between nodes are ensured by the Cypher operations used by the application. In practice, the application creates only allowed relationships such as `(:Character)-[:MEMBER_OF]->(:Gang)` and avoids invalid disconnected states through transactional writes.

Stored objects and replacement strategy

Neo4j does not use SQL stored procedures/functions/views in the same format as PostgreSQL. In this project, that functionality is replaced by Cypher queries

in the persistence layer and by service-layer logic in the Java application. Equivalent mechanisms include:

- Cypher[11] queries and reusable query templates
- graph projections and named queries at application layer
- APOC procedures (when enabled) for advanced utility workflows

If APOC is not enabled, reusable read/write behavior is still achieved by keeping the query logic in DAO/service classes instead of as stored database objects.

4.4. CRUD application for the graph database

The CRUD API can keep the same endpoint contract as the RDBMS solution. The key change is the persistence implementation:

- RDBMS: table-based DAO/JPA operations
- Neo4j: Cypher-based node/relationship operations

Important modeling difference:

- In SQL, many-to-many is handled by a junction table (`gang_affiliations`).
- In Neo4j, we do not create a junction table/node for this relation. We connect `Character` and `Gang` directly with `MEMBER_OF` and store membership metadata (for example `join_date`) on the relationship.

This keeps the graph model close to its core strength: relationships are first-class data.

From the client perspective, the CRUD flow can remain the same: the same HTTP endpoints, DTOs, and auth rules can be reused. The difference is that the graph version translates each request into Cypher that creates, matches, updates, or deletes nodes and relationships instead of rows in tables.

Example:

- **Create membership in RDBMS**: insert a row into `gang_affiliations(character_id, gang_id, join_date)`.
- **Create membership in Neo4j**: match the `Character` node and the `Gang` node, then create `(:Character)-[:MEMBER_OF {join_date: ...}]->(:Gang)`.

This means the application logic is largely identical to the RDBMS version, while the data layer and query language are the main parts that change.

4.5. API route overview

The HTTP route contract is now centralized through modular route classes:

- `Routes` (top-level composition)
- `AuthRoutes`

[11] Cypher Query Language Manual: https://neo4j.com/docs/cypher-manual/current/

- `ProfileRoutes`
- `AdminRoutes`
- `GameplayRoutes`

Each route class owns its own CRUD endpoints directly. `Routes` only collects them into one API tree. This route structure can stay the same for both the RDBMS and graph-database implementations, because the difference is mainly below the route layer in services/DAOs.

Route schema (current structure):

```
Routes
|-- /api/auth -> AuthRoutes
|   |-- POST /login
|   |-- POST /register
|   `-- POST /logout
|-- /api/profiles -> ProfileRoutes
|   |-- GET /            (ADMIN)
|   |-- POST /           (ADMIN)
|   `-- /{id}
|       |-- GET          (owner or ADMIN)
|       |-- PUT          (owner or ADMIN)
|       `-- DELETE       (owner or ADMIN)
|-- /api/drugs -> AdminRoutes (ADMIN CRUD)
|-- /api/quests -> AdminRoutes (ADMIN CRUD)
`-- /api/characters, /houses, /garages, /vehicles,
    /character-drug, /character-quest, /gangs,
    /gang-affiliations -> GameplayRoutes (authenticated CRUD)
```

Method	Endpoint	Access	Purpose
Authentication			
POST	/api/auth/login	Public	Authenticate by username and password.
POST	/api/auth/register	Public	Register a new profile with default ‘USER‘ role.
POST	/api/auth/logout	Public	Confirm logout in the stateless Basic Auth flow.
Profiles			
GET	/api/profiles	ADMIN	List all profiles.
POST	/api/profiles	ADMIN	Create a profile directly from the admin side.
GET	/api/profiles/{id}	Owner or ADMIN	Get one profile.
PUT	/api/profiles/{id}	Owner or ADMIN	Update one profile.
DELETE	/api/profiles/{id}	Owner or ADMIN	Delete one profile.

Method	Endpoint	Access	Purpose
Characters			
GET	/api/characters	Authenticated	List all characters.
POST	/api/characters	Authenticated	Create one character.
PUT	/api/characters/{id}	Authenticated	Update one character.
DELETE	/api/characters/{id}	Authenticated	Delete one character.
Houses			
GET	/api/houses	Authenticated	List all houses.
POST	/api/houses	Authenticated	Create a house.
GET	/api/houses/{id}	Authenticated	Get one house.
PUT	/api/houses/{id}	Authenticated	Update one house.
DELETE	/api/houses/{id}	Authenticated	Delete one house.
Garages			
GET	/api/garages	Authenticated	List all garages.
POST	/api/garages	Authenticated	Create a garage.
GET	/api/garages/{id}	Authenticated	Get one garage.
PUT	/api/garages/{id}	Authenticated	Update one garage.
DELETE	/api/garages/{id}	Authenticated	Delete one garage.
Vehicles			
GET	/api/vehicles	Authenticated	List all vehicles.
POST	/api/vehicles	Authenticated	Create a vehicle.
GET	/api/vehicles/{id}	Authenticated	Get one vehicle.
PUT	/api/vehicles/{id}	Authenticated	Update one vehicle.
DELETE	/api/vehicles/{id}	Authenticated	Delete one vehicle.
Character-Drug Links			
GET	/api/character-drug	Authenticated	List all character-drug links.
POST	/api/character-drug	Authenticated	Create a character-drug link.
GET	/api/character-drug/{id}	Authenticated	Get one character-drug link.
PUT	/api/character-drug/{id}	Authenticated	Update one character-drug link.
DELETE	/api/character-drug/{id}	Authenticated	Delete one character-drug link.
Character-Quest Links			
GET	/api/character-quest	Authenticated	List all character-quest links.
POST	/api/character-quest	Authenticated	Create a character-quest link.
GET	/api/character-quest/{id}	Authenticated	Get one character-quest link.
PUT	/api/character-quest/{id}	Authenticated	Update one character-quest link.
DELETE	/api/character-quest/{id}	Authenticated	Delete one character-quest link.
Gangs			
GET	/api/gangs	Authenticated	List all gangs.
POST	/api/gangs	Authenticated	Create a gang.
GET	/api/gangs/{id}	Authenticated	Get one gang.
PUT	/api/gangs/{id}	Authenticated	Update one gang.
DELETE	/api/gangs/{id}	Authenticated	Delete one gang.

Method	Endpoint	Access	Purpose
Gang Affiliations			
GET	/api/gang-affiliations	Authenticated	List all gang affiliations.
POST	/api/gang-affiliations	Authenticated	Create a gang affiliation.
GET	/api/gang-affiliations/{id}	Authenticated	Get one gang affiliation.
PUT	/api/gang-affiliations/{id}	Authenticated	Update one gang affiliation.
DELETE	/api/gang-affiliations/{id}	Authenticated	Delete one gang affiliation.
Drugs			
GET	/api/drugs	ADMIN	List all drugs.
POST	/api/drugs	ADMIN	Create a drug.
GET	/api/drugs/{id}	ADMIN	Get one drug.
PUT	/api/drugs/{id}	ADMIN	Update one drug.
DELETE	/api/drugs/{id}	ADMIN	Delete one drug.
Quests			
GET	/api/quests	ADMIN	List all quests.
POST	/api/quests	ADMIN	Create a quest.
GET	/api/quests/{id}	ADMIN	Get one quest.
PUT	/api/quests/{id}	ADMIN	Update one quest.
DELETE	/api/quests/{id}	ADMIN	Delete one quest.

5. Migration

The migration application migrates data from the relational PostgreSQL database into both MongoDB and Neo4j. PostgreSQL is treated as the source database because it has the strictest normalized model and the clearest integrity rules. The startup process, defined in `Main.java`, orchestrates the complete flow:

1. **Reset**: `PostgresReset.resetAndSeed(false)` truncates all PostgreSQL tables and reseeds them from `sqls/seed.sql`.
2. **Migrate**: `PostgresMigration.migrateAll(false)` loads the fresh PostgreSQL snapshot and transforms it into MongoDB and Neo4j.
3. **Persistence Setup**: Three `AppPersistence` instances are created via `PersistenceBootstrap.createPersistence()`, one for each database type (POSTGRES, MONGODB, NEO4J).
4. **HTTP Startup**: Three independent REST API instances are launched, each bound to a different port (7072 for PostgreSQL, 7073 for MongoDB, 7074 for Neo4j), each with their corresponding persistence layer.

The migration is split into small, focused classes:

- `PostgresReset` truncates and reseeds PostgreSQL from `seed.sql`.
- `PostgresMigration` coordinates the entire migration workflow.
- `PostgresSnapshotLoader` reads all source data from PostgreSQL into memory.

- `MigrationSnapshot` acts as a container for the loaded data during transformation.
- `MongoMigration` writes profile-centered documents and catalog collections to MongoDB.
- `Neo4jMigration` writes nodes and relationships to Neo4j.
- `EntityCopies` creates detached entity copies so the migration process does not depend on lazy-loaded JPA state.

For MongoDB, profile data becomes the main aggregate. Characters, houses, garages, vehicles, and relationship metadata are embedded inside profile documents, while drugs, quests, and gangs remain separate catalog collections. For Neo4j, relational foreign keys and junction tables are transformed into relationships such as `OWNS`, `HAS_HOUSE`, `HAS_GARAGE`, `STORES`, `HAS_DRUG`, `HAS_QUEST`, and `MEMBER_OF`.

This design ensures that PostgreSQL remains the authoritative source of truth during development, while MongoDB and Neo4j are always regenerated as fresh projections of the normalized relational data.

6. Discussion

The three database types model the same RPG domain in different ways. PostgreSQL is strongest when the goal is integrity, normalization, and transactional consistency. MongoDB is strongest when the application needs to load a complete profile aggregate in one document. Neo4j is strongest when the most important questions follow relationships, such as which gangs a character belongs to or which assets are connected to a character.

For data modeling, PostgreSQL uses normalized tables and junction tables. MongoDB uses embedded documents and references to catalog collections. Neo4j uses nodes and relationships, with relationship properties replacing some relational junction-table columns.

For transactions, PostgreSQL provides the clearest and strongest model for multi-table consistency. MongoDB can support transactions, but this project mainly uses aggregate-style writes. Neo4j supports transactional Cypher writes, which are useful when creating nodes and relationships together.

For constraints and integrity, PostgreSQL is the best fit because foreign keys, unique constraints, and check constraints are built into the schema. MongoDB and Neo4j require more responsibility in application code and repository logic. This is a trade-off: they give flexible models, but less automatic relational enforcement.

Overall, PostgreSQL fits the domain best as the primary source of truth, because the RPG data contains many mandatory relationships. MongoDB and Neo4j are valuable alternative views of the same data, optimized for different query patterns.

7. Reflection

The most difficult part was modeling the same domain three times without copying the relational design blindly into MongoDB and Neo4j. The team had to decide what should remain normalized, what should be embedded, and what should become a relationship.

Transactions and security were also challenging because the application uses shared route/controller logic while each database has a different persistence implementation. The solution was simplified by keeping the API contract stable and moving database-specific behavior into the repository layer.

One design mistake was adding too much abstraction and audit logic early in the project. This made the code harder to understand. The final version removes the audit feature and simplifies several layers, which makes the project easier to explain and maintain.

In practice, the project showed that database design is not only about storing data. It is about choosing where integrity should live, how data will be queried, and which compromises are acceptable for the domain.

8. Conclusion

The project implements one RPG game domain across PostgreSQL, MongoDB, and Neo4j. PostgreSQL provides the normalized relational source model, MongoDB provides a document-oriented profile aggregate, and Neo4j provides a relationship-focused graph model.

The Java application exposes CRUD endpoints through a shared REST API and uses database-specific repositories behind the same route structure. The migration application demonstrates how relational data can be transformed into both document and graph models.

The main insight is that no single database model is best for every problem. PostgreSQL is strongest for integrity, MongoDB for aggregate documents, and Neo4j for relationship traversal.

9. References

The report uses the following technical references:

- PostgreSQL Documentation. https://www.postgresql.org/docs/
- MongoDB Documentation. https://www.mongodb.com/docs/
- Neo4j Documentation. https://neo4j.com/docs/
- Cypher Query Language Manual. https://neo4j.com/docs/cypher-manual/current/
- Java 17 Documentation. https://docs.oracle.com/en/java/javase/17/
- Apache Maven Documentation. https://maven.apache.org/guides/
- Hibernate ORM Documentation. https://hibernate.org/orm/documentation/
- Javalin Documentation. https://javalin.io/documentation
- Project source repository. https://github.com/AhmadAlkaseb/Bajls

Appendix A. Links and file references

- Postgresql
- Java 17
- Maven
- Hibernate
- Javalin
- MongoDB
- Neo4j
- Cypher Manual
- Project Repository

File references used in report

- Docker-compose.yml
- Main.java
- Db/init.sql
- Schema.sql
- Daily_loyalty_bonus.sql
- Seed.sql
- ApplicationConfig.java
- Routes.java
- AuthRoutes.java
- ProfileRoutes.java
- AdminRoutes.java
- GameplayRoutes.java
- CrudController.java
- AuthService.java
- LoginRequestDTO.java
- LoginResponseDTO.java
- ProfileDTO.java
- JpaDao.java

- JpaReadDao.java
- ProfileDao.java
- GameCharacterDao.java
- DrugDao.java
- QuestDao.java
- MongoDB-design.json
- Neo4j-design.png
- Document3.png
- Graph-Database.png
- What-is-a-relational-database.jpg
- Conceptual-uml-03092026.puml
- Logical-uml-09032026.puml
- Physical-uml-03092026.puml
- View-v_character_overview.png
- View-v_gang_overview.png
- View-v_character_appearance.png